CORE JAVA TRAINING · PART 2 OF 25

Data Types & Operators

Predict exactly what every expression does, including the silent bugs

Difficulty	Easy–Medium
Prepared for	Mitansh Kanani
Starts with	Revision page: your Part 1 mistakes
Covers	variables · scope · final · var · 8 primitives · literals
	primitive vs reference · casting · promotion · overflow · floating point
	all operators · short-circuiting · bitwise flags · precedence · two’s complement
Assignments	8 coding tasks + 1 carried from Part 1
Tested on	JDK 21 (all code, outputs and error messages verified)

Contents

0	Revision: mistakes from Part 1
1	Variables
2	The 8 primitive types
3	Literals
4	Primitive types vs reference types
5	Type conversion and casting
6	Number traps
7	Operators
8	Under the hood: how numbers are stored
9	Common errors in this part
10	Self-check questions
11	Cheat sheet
12	Assignments

Part 2 is about the raw material every Java program is built from: **data** and the **operations** on it. It looks easy, and that is the trap. Most bugs in this topic compile perfectly and give a wrong answer silently. By the end, you should be able to predict exactly what any expression prints, including overflow, casting and the famous `x = x++`.

> **Note**
> Every output and error message in this PDF was compiled and run on JDK 21. Type the examples yourself, and use **jshell** for quick “what does this print?” checks.

0 Revision: mistakes from Part 1

Read this page before anything else. Each row is something you actually did in the Part 1 assignments, and the idea you should keep. These are the exact traps evaluators set.

What happened	What to remember
You thought integer division rounds or floors (A5).	Integer division **truncates toward zero**: it drops the decimal part. `19 / 5` is `3` (not 4), and `-17 / 5` is `-3` (floor would give -4).
You printed a double with `%d` and got `IllegalFormatConversionException` (A5).	The format specifier must match the type: `%d` for whole numbers, `%f` for decimals. javac does **not** check format strings, so this only fails at runtime.
Spaces around `\n` shifted your card lines (A1).	Everything after `\n` is printed at the start of the next line, including spaces.
You typed column spacing by hand; the `10` row went out of line (A6).	Let printf align for you. One format string for every row: `"%-15s%8d%11.2f%11.2f%n"`. Width specifiers pad automatically, whatever the value's length.
In ErrorLab, results for some cases didn't make sense (A7).	When testing, restore the original file, **recompile**, then make exactly one change. A leftover change from the previous case gives misleading errors.
You thought a class without `public` cannot be run (A7, case 10).	It runs. A non-public class just can't be accessed from other packages. The launcher only needs `main` itself to be `public static void`.
You thought `int main` gives "Main method not found" (A7, case 8).	On JDK 21 the message is `Main method must return a value of type void`. The launcher found main but rejected its return type.
You said Java invokes the default constructor "when we make a class" (A8).	The **compiler (javac)** adds a no-argument constructor only if you wrote none. It runs when an **object** is created (`new Lion()`), and its first job is calling the parent's constructor, `Object()`.
You skipped the "why" explanations.	Knowing *that* something fails is fresher level. Knowing *why* is what evaluations test. Write one sentence of why for every surprising result.

Carried-over assignment

Carried over from Part 1: the **Javadoc bonus** (`Documented.java`) is now assignment B1 in Section 12 of this part.

1 Variables

1.1 What a variable is

A **variable** is a named piece of memory that holds a value. Every variable in Java has three things: a **type** (what kind of value it can hold), a **name**, and a **value**. Because Java is statically typed, the type is fixed when you declare the variable and can never change.

Declaring variables

```
int age = 25;          // type: int    name: age    value: 25
double price = 99.5;   // type: double name: price  value: 99.5
char grade = 'A';      // type: char   name: grade  value: 'A'
boolean passed = true; // type: boolean
String name = "Mitansh"; // String is a class, not a primitive (Section 4)
```

1.2 Declare, initialize, assign

```
int count;          // declaration: reserve memory, no value yet
count = 10;         // assignment: put a value in
int total = 50;     // declaration + initialization in one line
total = total + 5;  // reassignment: read the old value, store a new one

int a = 1, b = 2, c = 3;   // several of the same type (allowed, but one per line is clearer)
```

Common mistake

`=` is **assignment**, not "equals". `total = total + 5` means: read total, add 5, store the result back in total. To *compare*, use `==`.

1.3 Local variables must be initialized

A variable declared inside a method is a **local variable**. Java refuses to let you read it before you have given it a value:

```
int x;
System.out.println(x);
```

javac

```
error: variable x might not have been initialized
        System.out.println(x);
                           ^
```

Variables declared directly inside a class (called **fields**, Part 7) are different: they get a **default value** automatically. You will see that table in Section 2.

Your evaluator might ask

Q: Why does Java force local variables to be initialized, but not fields?

Reading garbage memory is a classic C bug. The compiler can track every path through a method, so it can prove whether a local variable was set. Fields can be set from anywhere in the program, so the JVM zeroes them out when the object is created instead.

1.4 Scope: where a variable exists

A variable exists only from the line where it is declared to the end of the **block** (`{ }`) that contains it. This is called its **scope**.

S1.java

```
public class S1 {
    public static void main(String[] args) {
        int a = 10;
        {
            int b = 20;
            System.out.println(a + b);   // OK: a and b both visible
        }
        System.out.println(b);           // b no longer exists
    }
}
```

javac

```
S1.java:8: error: cannot find symbol
        System.out.println(b);
                           ^
  symbol:   variable b
```

You also cannot declare the same name again in an inner block while the outer one is still in scope: `error: variable a is already defined in method main(String[])`.

> **Under the hood**
> Scope is a **compile-time** idea. In bytecode, local variables are just numbered slots in the method's stack frame. When a block ends, its slot can be reused for the next variable. That is why scope costs nothing at runtime.

1.5 Constants with final

```
final int MAX_USERS = 100;
MAX_USERS = 200;
```

javac

```
error: cannot assign a value to final variable MAX_USERS
```

`final` means the variable can be assigned **exactly once**. By convention, constants use `UPPER_SNAKE_CASE`. Use them instead of "magic numbers": `price * GST_RATE` explains itself; `price * 0.18` does not.

1.6 var: local type inference (Java 10+)

```
var count = 10;          // compiler infers int
var name = "Mitansh";    // compiler infers String
var price = 99.5;        // compiler infers double
var x;                   // ERROR
```

```
javac

error: cannot infer type for local variable x
  (cannot use 'var' on variable without initializer)
```

- `var` does **not** make Java dynamically typed. The type is decided once, at compile time, from the value on the right. `count` is an int forever.
- Only for local variables with an initializer. Not for fields, method parameters or return types.
- Use it when the type is obvious from the right side. Avoid it when it hides the type: `var result = calculate();` forces the reader to go look.

> **Trainer's tip**
> If your company's code is on Java 8, `var` will not compile there. Know it, but match the version your team uses.

2 The 8 primitive types

Java has exactly eight **primitive** types. They hold simple values directly, are not objects, and their sizes are the **same on every platform** (unlike C, where int's size depends on the machine). That guarantee is part of "write once, run anywhere".

Type	Size	Range	Default*	Example
`byte`	8 bit	-128 to 127	0	`byte b = 100;`
`short`	16 bit	-32,768 to 32,767	0	`short s = 20000;`
`int`	32 bit	-2,147,483,648 to 2,147,483,647 (about ±2.1 billion)	0	`int i = 1_000_000;`
`long`	64 bit	about $\pm 9.2 \times 10^{18}$	0L	`long l = 9_000_000_000L;`
`float`	32 bit	about $\pm 3.4 \times 10^{38}$, ~7 significant digits	0.0f	`float f = 3.14f;`
`double`	64 bit	about $\pm 1.8 \times 10^{308}$, ~15–16 significant digits	0.0	`double d = 3.14;`
`char`	16 bit	0 to 65,535 (unsigned)	'\u0000'	`char c = 'A';`
`boolean`	not fixed	true or false only	false	`boolean ok = true;`

* Defaults apply to **fields** and array elements only. Local variables have no default (Section 1.3).

You never need to memorise ranges: ask the wrapper classes

```
System.out.println(Byte.MIN_VALUE + " " + Byte.MAX_VALUE);       // -128 127
System.out.println(Integer.MIN_VALUE + " " + Integer.MAX_VALUE); // -2147483648 2147483647
System.out.println(Long.MAX_VALUE);                              // 9223372036854775807
System.out.println(Double.MAX_VALUE);                            // 1.7976931348623157E308
System.out.println(Integer.SIZE + " " + Double.BYTES);           // 32 8
```

2.1 Integer types: byte, short, int, long

- `int` is the default choice for whole numbers. Plain whole-number literals like `42` are ints.
- Use `long` when values can pass about 2.1 billion: timestamps in milliseconds, file sizes, money in paise, population counts.
- `byte` is mostly used for raw data: file contents, network packets, encryption. You will see `byte[]` a lot in I/O (Part 18).
- `short` is rare in modern code.

Under the hood

Why is byte's range -128 to 127 and not -127 to 127? 8 bits give 256 patterns. Java uses **two's complement** (Section 8), which has only one zero, so one extra pattern goes to the negative side. The same logic gives int its lopsided range.

2.2 Floating-point types: float, double

- `double` is the default for decimals. A literal like `3.14` is a double.
- `float` has about half the precision. Use it only when memory really matters (graphics, huge arrays).
- Both are **approximate**. They cannot store most decimal fractions exactly (Section 6.2).

Common mistake

`Double.MIN_VALUE` is **not** the most negative double. It is the **smallest positive** double, `4.9E-324`. The most negative double is `-Double.MAX_VALUE`. Integer's MIN_VALUE does not work this way, which is why this trap catches people.

2.3 char

A `char` holds one 16-bit **UTF-16 code unit**: a number from 0 to 65,535 that represents a character. That is why it is 2 bytes in Java (C's char is 1 byte, ASCII only). Because a char is really a number, you can do arithmetic with it:

```
char c = 'A';
System.out.println((int) c);        // 65
System.out.println('A' + 1);        // 66   (char + int -> int)
System.out.println((char) ('A' + 1)); // B
System.out.println('a' + 'b');      // 195  (97 + 98, NOT "ab")
System.out.println((char) 97);      // a
```

Characters	Codes	Handy fact
'0' to '9'	48 to 57	`ch - '0'` turns a digit character into its number value.
'A' to 'Z'	65 to 90	Uppercase comes before lowercase.
'a' to 'z'	97 to 122	Lowercase = uppercase + 32.

Note

Some characters, such as many emoji, need **two** chars (a "surrogate pair"). So `String.length()` counts chars, not visible characters. You will meet this again in Part 6.

2.4 boolean

A `boolean` is only `true` or `false`. Unlike C, Java does **not** treat numbers as booleans:

```
boolean b = 1;
```

javac

```
error: incompatible types: int cannot be converted to boolean
```

Your evaluator might ask

Q: What is the size of a boolean?

The Java specification does not fix it. It represents one bit of information, but the JVM usually stores a boolean local as an int, and a boolean array element as one byte. The honest answer is "JVM-dependent".

3 Literals

A **literal** is a value written directly in the code, like `42`, `3.14`, `'A'` or `"Hi"`. Their exact form decides their type.

3.1 Integer literals

Form	Example	Value	Notes
Decimal	`42`	`42`	Normal numbers. Type is int.
Binary	`0b1010`	`10`	Prefix 0b. Java 7+.
Octal	`012`	`10`	Leading 0 means octal! A classic trap.
Hexadecimal	`0xFF`	`255`	Prefix 0x. Common for colors, bytes, flags.
Long	`3_000_000_000L`	`3000000000`	Suffix L (always uppercase; l looks like 1).
Underscores	`1_000_000`	`1000000`	Only for readability. Not at the start or end, not next to the dot.

```
int big = 3000000000;
```

javac

```
error: integer number too large
```

Without `L`, every whole-number literal is an int. 3 billion does not fit in an int, so the compiler stops you, even if you planned to store it in a long.

Common mistake

`int code = 012;` stores **10**, not 12. A leading zero makes it octal. Never pad numbers with zeros in code (for example, PIN codes or IDs). Store those as Strings.

3.2 Floating-point literals

```
double d1 = 3.14;      // double by default
double d2 = 1.5e3;     // scientific notation: 1500.0
float  f1 = 3.14f;     // suffix f makes it a float
float  f2 = 3.14;      // ERROR
```

```
javac

error: incompatible types: possible lossy conversion from double to float
```

`3.14` is a double (64 bits). Putting it into a float (32 bits) could lose precision, so Java demands that you say so explicitly with `f` or a cast.

3.3 char, boolean, String and null literals

```
char c1 = 'A';         // single quotes, exactly one character
char c2 = '\n';        // escape sequence
char c3 = '\u0041';    // Unicode escape: 'A'
char c4 = 65;          // also 'A': an int constant that fits is allowed
boolean ok = true;
String s = "Hello";    // double quotes = String
String nothing = null; // "no object". Only for reference types, never for primitives
```

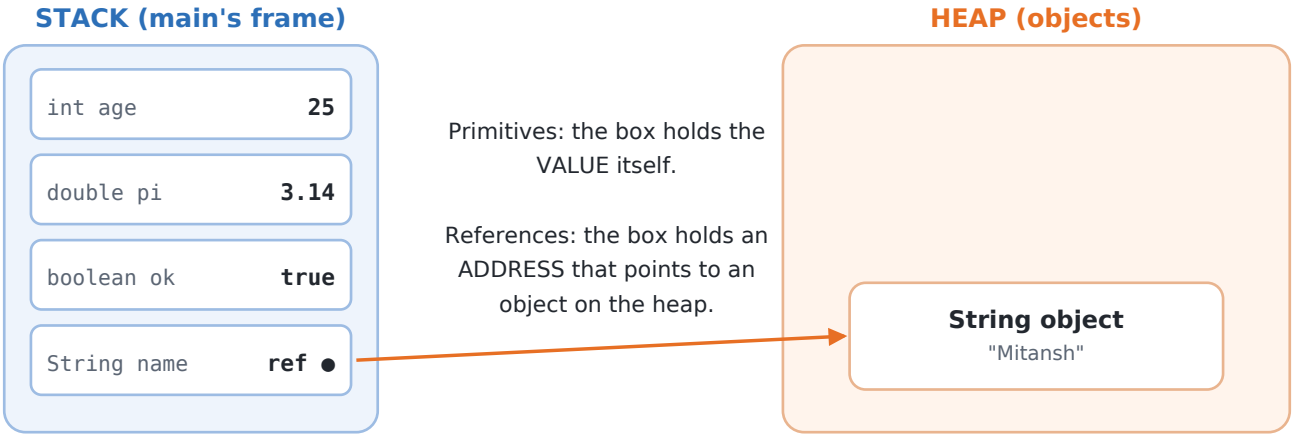
Common mistake

`'A'` (single quotes) is a char. `"A"` (double quotes) is a String. `'AB'` does not compile: a char holds exactly one character.

4 Primitive types vs reference types

Every type in Java is either one of the 8 primitives or a **reference type** (classes like `String`, arrays, interfaces, enums). The difference is **what the variable actually holds**.

```
int age = 25;
double pi = 3.14;
boolean ok = true;
String name = "Mitansh";
```



- A primitive variable stores the value itself in the method’s stack frame.
- A reference variable stores an **address** (a reference). The object itself lives on the **heap**.
- Reference variables can be `null` (pointing at nothing). Primitives can never be null.

- Primitives start with lowercase (`int`); class types start with uppercase (`String`).

Under the hood

This is the root of many future topics:

Part 5: why Java is pass-by-value even for objects (the address is copied).

Part 6: why `==` on two Strings can be false even when the text is the same (it compares addresses, not text).

Part 13: wrapper classes like `Integer` that wrap a primitive in an object.

Your evaluator might ask

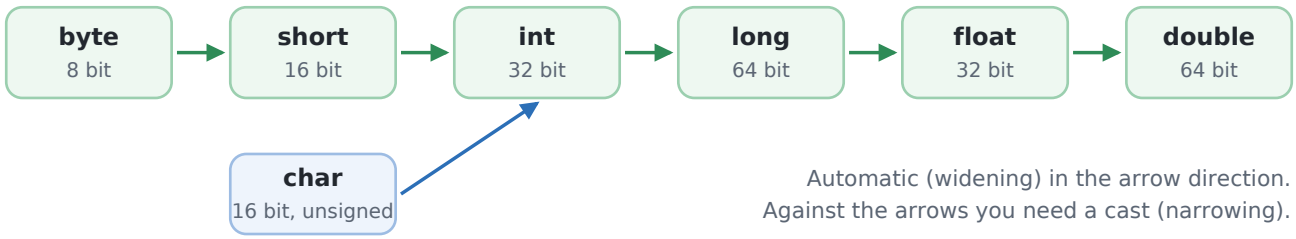
Q: Is String a primitive?

No. String is a class in java.lang. It feels like a primitive because Java gives it special support: double-quote literals, and `+` for joining.

5 Type conversion and casting

5.1 Widening: automatic and safe

Java automatically converts a value to a **bigger** type when no information can be lost (or, for int or long to float, when only precision can be lost). This is called **widening**.




```
int i = 100;
long l = i;        // int -> long, automatic
double d = l;      // long -> double, automatic
char c = 'A';
int code = c;      // char -> int, automatic: 65
```

Under the hood

int to float (and long to float or double) counts as “widening”: a float can *hold* a value that big, but with only about 7 digits of precision:

`float fb = 123456789;` then `(int) fb` gives `123456792`.

No error, no warning. The low digits are silently lost.

5.2 Narrowing: you must cast

Going the other way can lose data, so Java requires an explicit **cast**: the target type in brackets. A cast is you telling the compiler “I know, do it anyway”.

```
long y = 5;
int z = y;          // ERROR
int z = (int) y;    // OK: explicit cast
```

javac

```
error: incompatible types: possible lossy conversion from long to int
```

What actually happens when the value does not fit depends on the types:

Cast	Result	Why
`(byte) 130`	`-126`	Keeps only the lowest 8 bits. 130 is 10000010 in binary, and as a signed byte that pattern means -126.
`(byte) 300`	`44`	300 = 1 0010 1100. The low 8 bits are 0010 1100 = 44. The rest is thrown away.
`(byte) -129`	`127`	Wraps around the other way.
`(short) 70000`	`4464`	Keeps the low 16 bits.
`(int) 3.99`	`3`	double to int **truncates toward zero**. No rounding.
`(int) -3.99`	`-3`	Toward zero, so -3 (not -4).
`(int) 1e10`	`2147483647`	A double too big for int **saturates** at Integer.MAX_VALUE. It does not wrap.
`(long) 1e19`	`9223372036854775807`	Same: saturates at Long.MAX_VALUE.
`(int) Double.NaN`	`0`	NaN (“not a number”) becomes 0.
`(char) 66`	`B`	Number to character.

Your evaluator might ask

Q: What does (byte) 130 print, and why?

-126. Narrowing an integer keeps the lowest 8 bits. 130 is 10000010; the leftmost bit is the sign bit in two’s complement, so the value is 130 - 256 = -126.

Notice the difference: integer-to-integer narrowing **wraps** (keeps low bits), but decimal-to-integer narrowing **saturates** (clamps to the max or min).

5.3 Numeric promotion in expressions

This rule surprises everyone. Before any arithmetic, Java promotes the operands:

1. `byte`, `short` and `char` are always promoted to **int** first.
2. If either operand is `long`, `float` or `double`, the other is promoted to that type (the bigger one wins: double > float > long > int).
3. The result has that promoted type.

```
byte b = 10;
byte c = 20;
byte d = b + c;     // ERROR: b + c is an int!

char ch = 'A';
ch = ch + 1;        // ERROR: ch + 1 is an int
```

```
javac

error: incompatible types: possible lossy conversion from int to byte
error: incompatible types: possible lossy conversion from int to char
```

Fix it with a cast: `byte d = (byte) (b + c);`. Note the brackets around `b + c`. Without them, the cast would apply to b alone.

Under the hood

Why promote to int? The JVM has almost no instructions for byte or short arithmetic. It has `iadd` (int add), `ladd`, `fadd` and `dadd`. So small types are loaded as ints anyway.

Exception: `final byte e = 10; final byte f = 20; byte g = e + f;` compiles. Both are compile-time constants, so the compiler knows the result is 30 and that it fits in a byte. Same reason `byte h = 100;` works without a cast.

5.4 The hidden cast in compound assignment

```
byte b = 10;
b = b + 1;     // ERROR (int result)
b += 1;        // OK!
b++;           // OK!

int y = 10;
y += 5.7;      // compiles. y is now 15, not 15.7
```

Compound operators like `+=` include an automatic cast back to the variable's type. `b += 1` really means `b = (byte) (b + 1)`. Convenient, but it can also hide data loss silently, like `y += 5.7` dropping the .7.

Your evaluator might ask

Q: Why does b = b + 1 fail but b += 1 compile when b is a byte?

`b + 1` is promoted to int, and assigning an int to a byte needs a cast. Compound assignment operators perform that cast implicitly, as defined by the Java Language Specification.

5.5 Integer division and order of operations

You learned in Part 1 that int / int truncates. Combined with left-to-right evaluation, **the order** of the same operations changes the answer:

Same numbers, different answers

```
System.out.println(1 / 3 * 3);       // 0    (1/3 = 0, then 0*3)
System.out.println(1.0 / 3 * 3);     // 1.0
System.out.println(9 / 5 * 100);     // 100  (9/5 = 1)
System.out.println(100 * 9 / 5);     // 180

int c = 37;
System.out.println(9 / 5 * c + 32);  // 69    wrong: 9/5 became 1
System.out.println(c * 9 / 5 + 32);  // 98    better, still truncated
System.out.println(c * 9.0 / 5 + 32);// 98.6  correct
```

Trainer's tip

When a formula has a fraction like 9/5, make one operand a double (`9.0`) or cast before dividing: `(double) a / b`. Note: `(double) (a / b)` is wrong. It divides as ints first, then converts the already-truncated result.

6 Number traps

6.1 Integer overflow

When an int calculation goes past `Integer.MAX_VALUE`, Java does **not** throw an error. The value silently **wraps around** to the negative side, like a car odometer rolling over.

```
System.out.println(Integer.MAX_VALUE + 1);   // -2147483648
System.out.println(Integer.MIN_VALUE - 1);   //  2147483647

byte b = 127;
b++;
System.out.println(b);                       // -128
```

A real-world overflow bug

Milliseconds in one year

```
System.out.println(365 * 24 * 60 * 60 * 1000);   // 1471228928   WRONG
System.out.println(365L * 24 * 60 * 60 * 1000);  // 31536000000  correct
```

Every literal is an int, so the whole multiplication happens in int and overflows before anything else. Making the **first** operand a long (`365L`) makes every step long arithmetic.

```
int a = 1_000_000;
long bad  = a * a;          // -727379968   (multiplied as int, overflowed, THEN widened)
long good = (long) a * a;   // 1000000000000
```

Common mistake

Storing the result in a `long` does not help. The type of the **calculation** is decided by the operands, not by the variable on the left. Cast before the operation, not after.

Want a crash instead of a silent wrong answer?

```
System.out.println(Math.multiplyExact(2_000_000_000, 2));
```

Terminal

```
Exception in thread "main" java.lang.ArithmeticException: integer overflow
```

`Math.addExact`, `Math.subtractExact` and `Math.multiplyExact` throw an exception on overflow. Use them where a wrong number is worse than a crash, such as money, counts or sizes.

Your evaluator might ask

Q: What is Math.abs(Integer.MIN_VALUE)?

`-2147483648`. The positive version (2147483648) does not fit in an int, so it overflows back to MIN_VALUE. The absolute value of a number is negative here.

6.2 Floating-point imprecision

```
System.out.println(0.1 + 0.2);          // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3);   // false
System.out.println(1.1 * 3);            // 3.3000000000000003

float f = 0.1f;
System.out.println((double) f);         // 0.10000000149011612
```

This is not a Java bug. Computers store decimals in **binary**, and 0.1 has no exact binary form, just as 1/3 has no exact decimal form (0.3333...). The stored value is the closest possible approximation, and small errors add up (Section 8.2 shows why).

- **Never compare doubles with ==.** Check if they are close enough: `Math.abs(a - b) < 1e-9`.
- **Never use double or float for money.** Use `long` in the smallest unit (paise), or `BigDecimal` (Part 13).
- `10 == 10.0` is `true`: the int is promoted to double and these are exact. The problem is only with fractions that cannot be represented exactly.

6.3 Division by zero

Expression	Result
`10 / 0`	`ArithmeticException: / by zero (runtime crash)`
`10 % 0`	`ArithmeticException: / by zero`
`10.0 / 0`	`Infinity`
`-10.0 / 0`	`-Infinity`
`0.0 / 0`	`NaN (Not a Number)`
`10.0 % 0`	`NaN`

Integers have no way to represent infinity, so integer division by zero crashes. Doubles follow the IEEE 754 standard, which has special values for infinity and NaN, so they never crash.

Common mistake

`NaN == NaN` is `false`. NaN is not equal to anything, not even itself. To check for it, use `Double.isNaN(x)`.

7 Operators

7.1 Arithmetic: + - * / %

You know these already. The part people get wrong is `%` (remainder) with negative numbers and decimals:

```
System.out.println(17 % 5);    //  2
System.out.println(-17 % 5);   // -2   the sign follows the LEFT operand
System.out.println(17 % -5);   //  2
System.out.println(5.5 % 2);   //  1.5 works with decimals too
```

Common mistake

Because of negative remainders, `n % 2 == 1` is a **wrong** odd check: for -3 it gives -1, so it says -3 is not odd. Use `n % 2 != 0` instead.

7.2 Increment and decrement: ++ and --

`++x` (**pre**-increment) adds 1, then gives the **new** value. `x++` (**post**-increment) gives the **old** value, then adds 1. On their own line, both just add 1. The difference only matters inside a bigger expression.

```
int a = 5;
int b = a++;   // b = 5, a = 6   (use, then increment)
int c = ++a;   // a = 7, c = 7   (increment, then use)
```

Tracing a tricky expression

```
int i = 5;
int r = i++ + ++i;
```

Step	What happens	i	Running value
1	`i++`: use the old value, then add 1	6	5
2	`++i`: add 1, then use the new value	7	5 + 7
3	Add the two values	7	r = 12

The famous trap: x = x++

```
int x = 5;
x = x++;
System.out.println(x);   // 5 !
```

1. Evaluate the right side: `x++` saves the old value (5) as the result.
2. The increment happens: x becomes 6.
3. The assignment runs last: x is set to the saved result, 5. The increment is overwritten.

> **Trainer's tip**
> In real code, never use ++ or -- inside a bigger expression. Put them on their own line. These puzzles exist to test whether you understand evaluation order, and interviewers love them.

7.3 Assignment and compound assignment

Operator	Example	Same as (with hidden cast)
`=`	`x = 5`	—
`+= -= *= /= %=`	`x += 3`	`x = (type) (x + 3)`
`&= |= ^=`	`flags |= WRITE`	`flags = flags | WRITE`
`<<= >>= >>>=`	`x <<= 2`	`x = x << 2`

Assignment is **right-associative** and is itself an expression, so `p = q = 5;` first sets q to 5, then sets p to the result (5).

7.4 Relational: == != < > <= >=

These compare two values and always produce a `boolean`. `'A' == 65` is `true` (char promoted to int). For objects like Strings, `==` compares **references**, not content. That is Part 6's biggest lesson.

7.5 Logical: && || ! and short-circuiting

Operator	Meaning	Short-circuits?
`&&`	AND: true only if both are true	Yes: if the left is false, the right is never evaluated
`||`	OR: true if at least one is true	Yes: if the left is true, the right is never evaluated
`!`	NOT: flips true and false	—
`& |`	AND / OR on booleans	No: both sides are always evaluated
`^`	XOR: true if exactly one is true	No

Proof of short-circuiting

```
int c = 0;
boolean t = false && (++c > 0);
System.out.println(c);   // 0  right side never ran

c = 0;
t = false & (++c > 0);
System.out.println(c);   // 1  right side ran
```

Short-circuiting is not just an optimisation. It is how you safely guard risky checks:

```
if (name != null && name.length() > 0) { ... }   // safe: length() only runs if name is not null
if (count != 0 && total / count > 10) { ... }    // safe: no division by zero
```

> **Note**
> You will learn `if` properly in Part 3. Here, just notice that the order of the conditions matters.

7.6 Bitwise operators and shifts

These work on the individual **bits** of int and long values. Example with 5 (0101) and 3 (0011):

Operator	Name	Rule per bit	5 op 3	Result
`&`	AND	1 only if both bits are 1	`0101 & 0011`	`0001 = 1`
`|`	OR	1 if either bit is 1	`0101 | 0011`	`0111 = 7`
`^`	XOR	1 if the bits differ	`0101 ^ 0011`	`0110 = 6`
`~`	NOT	flips every bit	`~5`	`-6`
`<<`	left shift	shift left, fill with 0	`5 << 1`	`10 (×2)`
`>>`	signed right shift	shift right, copy the sign bit	`-16 >> 2`	`-4`
`>>>`	unsigned right shift	shift right, fill with 0	`-16 >>> 28`	`15`

Where bitwise operators are actually used

Permission flags: many yes/no settings packed into one int

```
int READ = 1, WRITE = 2, PRINT = 4, SHARE = 8;   // each is one bit: 0001 0010 0100 1000

int perms = READ | PRINT;                 // 5 (0101): can read and print
boolean canPrint = (perms & PRINT) != 0;  // true:  check a permission
boolean canWrite = (perms & WRITE) != 0;  // false
perms = perms | WRITE;                    // 7: grant write
perms = perms & ~PRINT;                   // 3: revoke print
```

This pattern is used in file permissions, security and rights-management products, network protocols and Java's own APIs. Other everyday uses:

- `(n & 1) == 0` checks if n is even (the last bit of an even number is 0).
- `n << k` multiplies by 2^k; `n >> k` divides by 2^k (rounding toward negative infinity for negatives).
- `n > 0 && (n & (n - 1)) == 0` checks if n is a power of two.

7.7 The ternary operator ? :

```
int max = a > b ? a : b;              // condition ? valueIfTrue : valueIfFalse
String result = marks >= 40 ? "Pass" : "Fail";

int max3 = p > q ? (p > r ? p : r) : (q > r ? q : r);   // nested: works, but hard to read
```

The ternary operator is an **expression**: it produces a value. Use it for simple either/or values. For anything bigger, use `if` (Part 3).

7.8 Precedence and associativity

When an expression has several operators, **precedence** decides which runs first, and **associativity** decides the order among operators of equal precedence.

Level (high to low)	Operators	Associativity
Postfix	`x++ x--`	left to right
Unary	`++x --x +x -x ! ~ (cast)`	right to left
Multiplicative	`* / %`	left to right
Additive	`+ -`	left to right
Shift	`<< >> >>>`	left to right
Relational	`< > <= >= instanceof`	left to right
Equality	`== !=`	left to right
Bitwise AND / XOR / OR	`& then ^ then |`	left to right
Logical AND	`&&`	left to right
Logical OR	`||`	left to right
Ternary	`? :`	right to left
Assignment	`= += -= etc.`	right to left

```
2 + 3 * 4                  // 14    (* before +)
10 - 4 - 3                 // 3     (left to right: (10-4)-3)
100 / 10 / 5               // 2     ((100/10)/5)
!(5 > 3) || 2 < 4 && false // false (&& before ||: false || (true && false))
```

Trainer's tip

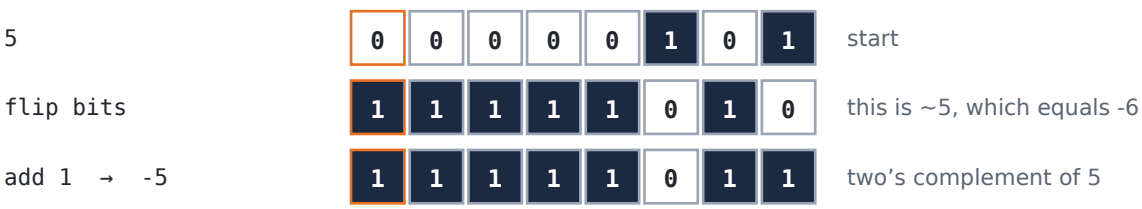
You do not need to memorise the whole table. Remember: unary, then * / %, then + -, then comparisons, then && before ||, then assignment last. When in doubt, **add parentheses**. Code is read far more often than written.

8 Under the hood: how numbers are stored

8.1 Two's complement: negative integers

Java stores negative integers in **two's complement**. The leftmost bit is the **sign bit** (0 = positive, 1 = negative). To get -x from x: **flip every bit, then add 1**.

Making -5 in 8 bits (byte). The orange-bordered box is the sign bit.



Why overflow wraps around



- There is only one zero (00000000), so the negative side gets one extra value: -128 to 127.
- Overflow is just binary addition carrying into the sign bit. No error, just a different meaning.
- `~x` always equals `-x - 1`. That is why `~5` is `-6`.
- `>>` copies the sign bit into the empty spaces, so negatives stay negative. `>>>` fills with 0, so a negative number becomes a big positive one.

See the bits yourself

```
System.out.println(Integer.toBinaryString(5));    // 101
System.out.println(Integer.toBinaryString(-5));   // 11111111111111111111111111111011
System.out.println(Integer.toHexString(255));     // ff
```

8.2 IEEE 754: how float and double are stored

A double is stored like scientific notation in binary: **sign** (1 bit), **exponent** (11 bits) and **mantissa** (52 bits), meaning ±mantissa × 2^{exponent}. A float uses 1 + 8 + 23 bits.

- The mantissa has a fixed number of bits, so precision is limited: about 15–16 decimal digits for double, about 7 for float.
- 0.1 in binary is 0.0001100110011001100110011... repeating forever. It is cut off to fit, so the stored value is slightly off. 0.1 + 0.2 adds two slightly-off values.
- The format has special patterns for +Infinity, -Infinity and NaN. That is why double division by zero does not crash.
- Large doubles can represent huge numbers but with gaps between them. That is why long to float or double can lose the last digits.

Your evaluator might ask

Q: Why is 0.1 + 0.2 not equal to 0.3 in Java?

doubles use binary IEEE 754 floating point. 0.1, 0.2 and 0.3 have no exact binary representation, so each is stored as the nearest approximation. The rounding errors of 0.1 and 0.2 add up to a result slightly different from the approximation of 0.3. Compare with a tolerance, or use BigDecimal for exact decimal values.

9 Common errors in this part (real messages)

You wrote	You see	Fix
`int x; println(x);`	`variable x might not have been initialized`	Give it a value first.
`int big = 3000000000;`	`integer number too large`	Use long with L: 3_000_000_000L.
`float f = 3.14;`	`incompatible types: possible lossy conversion from double to float`	3.14f or (float) 3.14.
`long y = 5; int z = y;`	`possible lossy conversion from long to int`	(int) y, if you are sure it fits.
`byte d = b + c;`	`possible lossy conversion from int to byte`	(byte) (b + c). Arithmetic makes an int.
`char c = 'A'; c = c + 1;`	`possible lossy conversion from int to char`	c++ or c += 1, or cast.
`boolean b = 1;`	`int cannot be converted to boolean`	Use true / false.
`var x;`	`cannot infer type for local variable x`	var needs an initializer.
`final int MAX = 10; MAX = 20;`	`cannot assign a value to final variable MAX`	Don’t reassign constants.
`Variable used outside its block`	`cannot find symbol symbol: variable b`	Declare it in the outer block.
`Same name in an inner block`	`variable a is already defined in method main(String[])`	Use a different name.
`10 / 0`	`ArithmeticException: / by zero` (runtime)	Check the divisor first.
`Math.multiplyExact overflow`	`ArithmeticException: integer overflow` (runtime)	Use long, or handle it.

Common mistake

The most dangerous problems in this part produce **no error at all**: overflow, integer division, lossy casts, `+=` truncation, and floating-point rounding. The compiler is happy; the answer is wrong. That is why you must be able to predict values in your head.

10 Self-check questions

Answer without looking, then check. If you miss one, re-read the section in brackets.

#	Question	Answer
1	Why must local variables be initialized but fields need not be?	The compiler can prove whether a local was assigned on every path; fields get default values when the object is created. (1.3)
2	Is var dynamic typing?	No. The type is inferred once at compile time and never changes. (1.6)
3	Why is byte’s range -128 to 127?	8 bits = 256 patterns; two’s complement has one zero, so the negative side gets one extra value. (2.1, 8.1)
4	What is Double.MIN_VALUE?	The smallest positive double (4.9E-324), not the most negative. (2.2)
5	What does 'a' + 'b' print? And "" + 'a' + 'b'?	195 (char + char = int). "ab" (String concatenation). (2.3)
6	What value does int x = 012; hold?	10. A leading 0 means octal. (3.1)
7	Why does float f = 3.14; fail?	3.14 is a double literal; double to float is narrowing. Use 3.14f. (3.2)
8	What does a reference variable actually store?	An address pointing to an object on the heap, not the object itself. (4)
9	(byte) 300 and (int) 1e10: what are they, and why are the rules different?	44 (integer narrowing keeps low bits, wraps) and 2147483647 (decimal to int saturates at the max). (5.2)
10	Why does byte d = b + c; fail when b and c are bytes?	byte/short/char are promoted to int before arithmetic, so b + c is an int. (5.3)
11	byte b = 10; Why does b += 1 compile but b = b + 1 not?	Compound assignment includes an implicit cast back to byte. (5.4)
12	Why does long bad = a * a; overflow when a is an int?	The multiplication is done in int before widening. Cast first: (long) a * a. (6.1)
13	Why is 0.1 + 0.2 != 0.3? How should you compare doubles?	Binary floating point can’t represent them exactly. Use Math.abs(a - b) < tolerance. (6.2, 8.2)
14	10 / 0 vs 10.0 / 0?	ArithmeticException vs Infinity. (6.3)
15	What is -17 % 5?	-2. The remainder takes the sign of the left operand. (7.1)
16	int x = 5; x = x++; what is x?	5. The old value is saved, x becomes 6, then the assignment writes 5 back. (7.2)
17	Difference between && and &?	&& short-circuits (skips the right side if the left is false); & always evaluates both. (7.5)
18	How do you check if a permission bit WRITE is set in flags?	(flags & WRITE) != 0. (7.6)

#	Question	Answer
19	Why is ~5 equal to -6?	~x flips all bits, which in two's complement equals -x - 1. (8.1)
20	What is the difference between >> and >>>?	>> fills with the sign bit (keeps negatives negative); >>> fills with 0. (7.6, 8.1)

11 Cheat sheet

Part 2 at a glance

```
TYPES      byte 8 | short 16 | int 32 | long 64 | float 32 | double 64
           char 16 (unsigned) | boolean (true/false only)
DEFAULTS   whole numbers -> int, decimals -> double
           fields only: 0 / 0.0 / false / '\u0000' / null   (locals: none!)
LITERALS   3_000_000_000L  3.14f  1.5e3  0b1010  012 (=10, octal!)  0xFF  '\u0041'

WIDENING   byte > short > int > long > float > double  (char > int)  automatic
NARROWING  cast needed: (int) 3.99 = 3   (byte) 130 = -126
           (int) 1e10 = 2147483647  (decimal -> int saturates, int -> byte wraps)
PROMOTION  byte/short/char in arithmetic -> int.  Bigger type wins.
COMPOUND   b += 1  is  b = (byte)(b + 1)   hidden cast, can hide data loss

OVERFLOW   Integer.MAX_VALUE + 1 == Integer.MIN_VALUE   (no error!)
           cast BEFORE multiplying: (long) a * a   or start with 365L * ...
           Math.addExact / multiplyExact throw on overflow
FLOATS     never ==, use Math.abs(a - b) < 1e-9.   Never double for money.
DIV BY 0   int -> ArithmeticException    double -> Infinity / NaN   NaN != NaN

++ / --    x++ use then add   ++x add then use   x = x++ leaves x unchanged
%          sign follows left operand: -17 % 5 == -2   odd: n % 2 != 0
LOGICAL    && || short-circuit       & | ^ always evaluate both sides
BITWISE    & | ^ ~ << >> >>>    even: (n & 1) == 0    set? (f & FLAG) != 0
           add: f |= FLAG    remove: f &= ~FLAG    toggle: f ^= FLAG
TERNARY    cond ? a : b
PRECEDENCE unary > * / % > + - > shifts > < > > == != > & ^ | > && > || > ?: > =
```

12 Assignments

Same rules as Part 1: no AI, no copy-paste, compile and run from the terminal, put everything in `part-02`. Add a one-line comment at the top of each file. For prediction tasks, write the prediction **first**, then run, then explain every wrong one.

Built from your revision page

These assignments deliberately reuse your Part 1 mistakes: printf widths (A1), truncation (A4), and the "why" habit (A2, A5).

A1. TypeTable.java (Easy)

Print this table using the wrapper constants (`Byte.SIZE`, `Integer.MIN_VALUE` and so on). **Do not type any number by hand.** Use **one** format string for every row, with widths. For char, cast its min and max to int. For float and double, print `-MAX_VALUE` as the minimum.

Expected output

```
TYPE       BITS                     MIN                     MAX
byte          8                    -128                     127
short        16                  -32768                   32767
int          32              -2147483648              2147483647
long         64    -9223372036854775808     9223372036854775807
float        32            -3.4028235E38             3.4028235E38
double       64 -1.7976931348623157E308  1.7976931348623157E308
char         16                       0                   65535
boolean       -                   false                    true
```

Hint: `%s` can print any value, including numbers.

A2. CastingLab.java (Easy–Medium)

Print each expression on its own line. Write your **prediction** as a comment first, then run. For every wrong prediction, add a comment explaining why.

14 expressions

```
(byte) 200          (byte) -130          (short) 40000
(int) 7.99          (int) -7.99          (int) 'z'
(char) 98           (int) 3e9            (int) 2_147_483_648L
(byte) (100 + 100)  'a' + 1              (char) ('a' + 1)
5 / 2 * 2.0         5 / 2.0 * 2
```

Then answer in a comment: `(int) 3e9` and `(int) 2_147_483_648L` both start with a value that is too big for an int. Why are the results completely different?

A3. NumberTraps.java (Medium)

Demonstrate three silent bugs and their fixes. Each part prints the wrong result, then the correct one.

1. **Overflow:** milliseconds in 30 days. Compute it with ints (wrong), then fix it with a long literal. Also print `Math.abs(Integer.MIN_VALUE)`.
2. **Money:** 3 items costing ₹1.10 each. Show that `1.1 * 3` with double is wrong. Then do it with a `long` in paise and print `Rs 3.30` using printf with `/ 100` and `% 100`. (Hint: `%02d` pads with a zero.)
3. **Crash instead of silence:** as the **last** line, use `Math.multiplyExact` on two ints that overflow, so the program ends with an `ArithmeticException`.

In comments, explain why the overflow happens even though 30 days of milliseconds fits in a long.

A4. TemperatureConverter.java (Easy)

Take a Celsius value as a command-line argument (it may have decimals). Print Fahrenheit and Kelvin with 2 decimal places, aligned with printf. Then show the integer bug.

Terminal

```
> java TemperatureConverter 37
Celsius    : 37.00
Fahrenheit : 98.60
Kelvin     : 310.15
Buggy int formula 9 / 5 * c + 32 gives: 69
```

F = C × 9/5 + 32 and K = C + 273.15. For the last line, convert the argument to an int using a cast from the double. In a comment, explain exactly why 69 is printed.

A5. IncrementPuzzle.java (Medium)

Predict every value **before** running. Then print `a, b, c, d, e, g, h` on one line. For each one, add a comment tracing how it was calculated step by step, like the table in Section 7.2.

The puzzle

```
int a = 5;
int b = a++ + a++;
int c = ++a + ++a;
int d = a-- - --a;
int e = 10;  e = e++ + ++e;
int g = 3;   g += g++ * 2;
int h = 2;   h = h++ * ++h;
```

A6. BitwiseToolkit.java (Medium)

Part 1: take a number n as an argument and print the following using **only** bitwise operators for the logic (no `%`, `*` or `/`, and no if):

Terminal

```
> java BitwiseToolkit 40
Number        : 40
Binary        : 101000
Is even       : true
n * 8 (<<)    : 320
n / 4 (>>)    : 10
Power of two  : false
~n            : -41
```

Part 2: define four permission constants: `READ = 1, WRITE = 2, PRINT = 4, SHARE = 8`. Start with a user who has READ and SHARE. Print whether each of the four permissions is set. Then grant PRINT, revoke READ, and print the final value in decimal and binary (expected: `12` and `1100`).

A7. SwapThreeWays.java (Easy–Medium)

Take two ints as arguments and swap them three ways, printing the values after each: (1) with a temporary variable, (2) with only `+` and `-`, (3) with only `^`. After each swap, swap back before the next method.

Then run it with `2147483647 1`. The arithmetic way overflows in the middle, but the final answer is still correct. Explain why in a comment. (Hint: Section 8.1.)

A8. ShortCircuitProof.java (Easy–Medium)

Using a counter variable that you increment inside the conditions, prove all four of these, printing the counter after each:

- `&&` skips the right side when the left is false.
- `&` evaluates both sides.

- `||` skips the right side when the left is true.
- `|` evaluates both sides.

Then take three ints as arguments and print the largest using a **nested ternary**, and print whether the largest is “even” or “odd” using another ternary.

B1. Documented.java **(Carried from Part 1)**

Write a class with a Javadoc comment on the class and on main (use `@author` and `@param`). Generate the docs with `javadoc -d docs Documented.java` and open `docs/index.html`. Add a screenshot to the repo. Add `docs/` to `.gitignore` so the generated folder is not committed.

> **What happens next**
> When you push Part 2, send the repo link. I will compile and run everything, and only list the mistakes worth fixing.